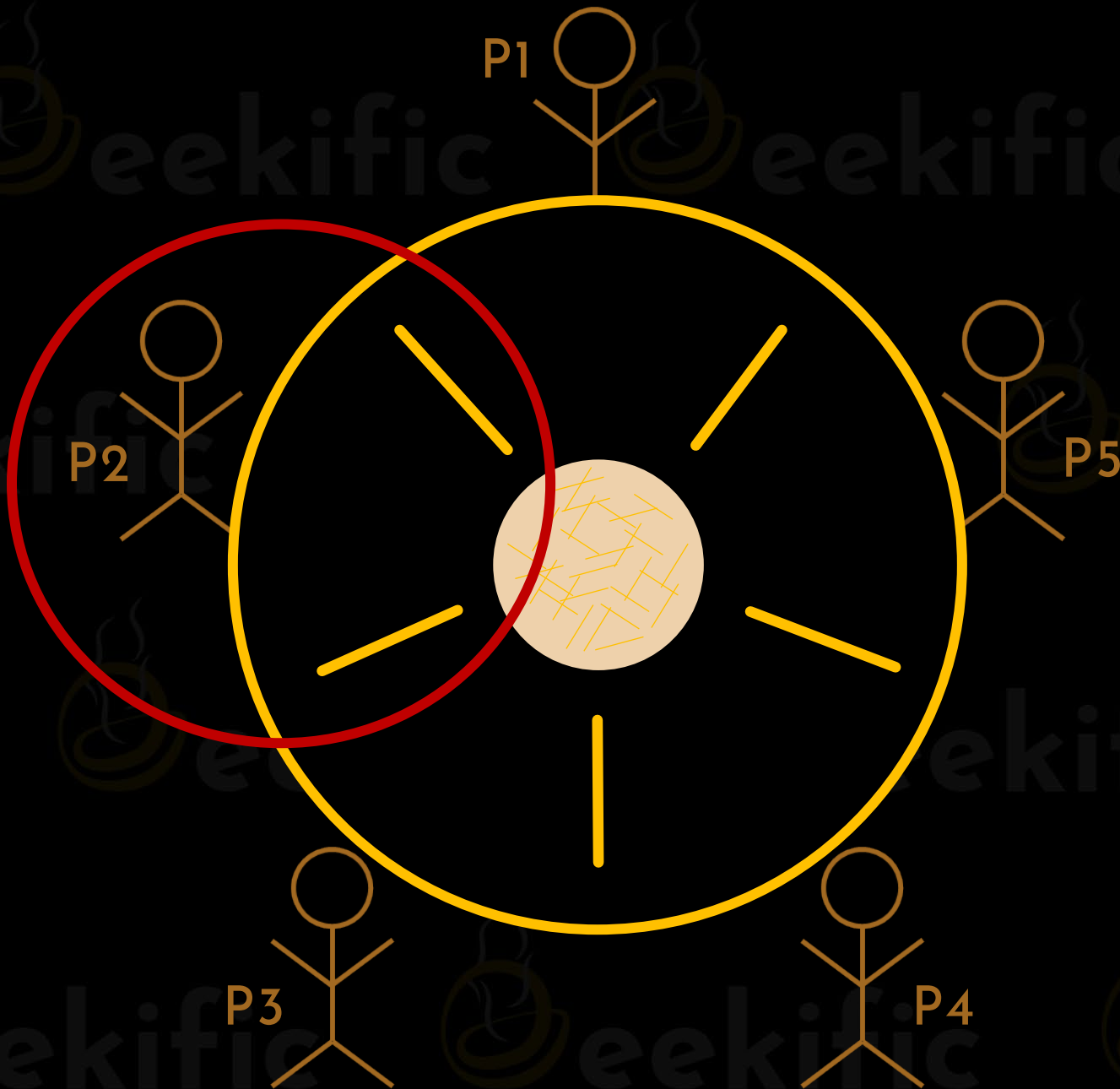


The Dining Philosophers

originally formulated in
1965 by Edsger Dijkstra





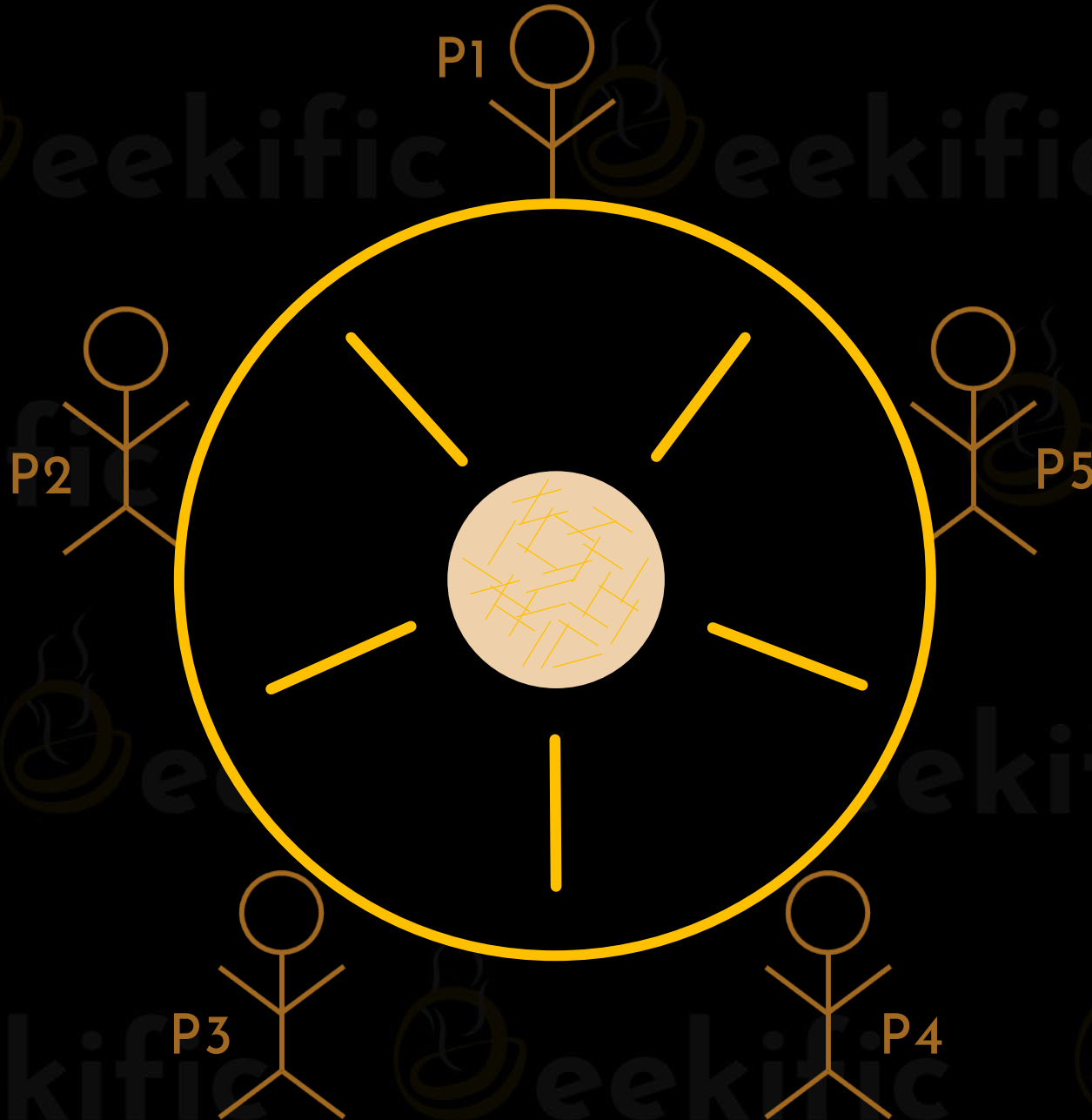
Philosophers can
either think or eat

to eat, a philosopher
needs both their right
and left chopstick

if not available
the philosopher
starts thinking

after they're done
eating, they will put
down both chopsticks





design a concurrent algorithm such that no philosopher will starve

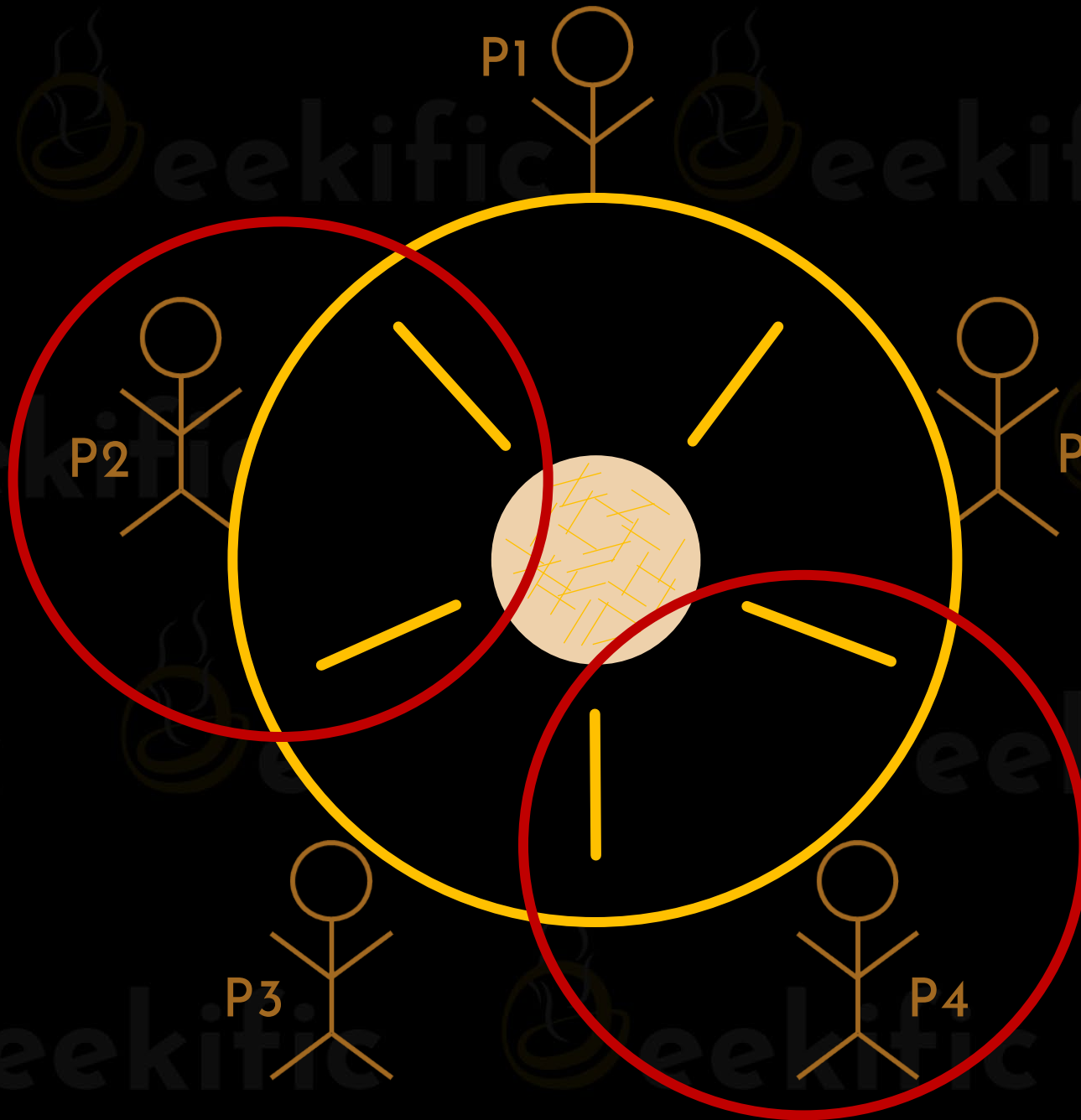
each of them can forever continue to alternate between eating and thinking
no philosopher can be aware of when others may want to eat or think



Race Conditions & Deadlocks

```
think until left chopstick is available; when it is, pick it up;  
think until right chopstick is available; when it is, pick it up;  
when both chopsticks are held, eat for a fixed amount of time;  
put the left chopstick down;  
put the right chopstick down;  
repeat from the beginning;
```





a **race condition** is when two threads, attempt to access the same resource at the same time

nothing guarantees that different philosophers will pick these chopsticks

even if each philosopher waits a different random time, one unlucky philosopher may never have a chance to eat



Deadlocks

suppose all philosophers held the left chopstick thinking, staring at the right side of the plate, unable to eat because there is no right chopstick, they will eventually starve

a race condition produces incorrect results, whereas a deadlocked process never makes any progress

it's a process waiting for a resource held by a second process that's waiting for a resource that the first holds



Race Condition



the thread that writes its value last, is preserved

the most common symptom is unpredictable values of shared variables

Deadlock



each thread is holding the variable the other wants

because of deadlocks programs will stop responding
this is known as a hang



```
public class Philosopher extends Thread {  
  
    private final Object left;  
    private final Object right;  
  
    Philosopher(Object left, Object right, int number) {  
        this.left = left;  
        this.right = right;  
        this.setName("Philosopher " + number);  
    }  
  
    ...  
    ...  
}
```



```

public class Philosopher extends Thread {
    ...

    @Override
    public void run() {
        try {
            while (true) {
                act(" is thinking");
                synchronized (left) {
                    act(" picked up the left chopstick");
                    synchronized (right) {
                        act(" picked up the right chopstick and is eating");
                    }
                    act(" put down both chopsticks, stopped eating and is thinking");
                }
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void act(String message) throws InterruptedException {
        System.out.println(Thread.currentThread().getName() + message);
        Thread.sleep(((int) (Math.random() * 100)));
    }
}

```



```

public class MainApp {

    private static final int NBR_OF_PHILOSOPHERS = 5;

    public static void main(String[] args) {

        Philosopher[] philosophers = new Philosopher[NBR_OF_PHILOSOPHERS];
        Object[] chopsticks = IntStream.range(0, NBR_OF_PHILOSOPHERS).mapToObj(i -> new Object()).toArray();

        ExecutorService executor = Executors.newFixedThreadPool(NBR_OF_PHILOSOPHERS);
        IntStream.range(0, NBR_OF_PHILOSOPHERS).forEach(i -> executor.submit(() -> {
            Object leftChopstick = chopsticks[i];
            Object rightChopstick = chopsticks[(i + 1) % NBR_OF_PHILOSOPHERS];

            philosophers[i] = new Philosopher(leftChopstick, rightChopstick, (i + 1));
            philosophers[i].start();
        }));
    }
}

```

Philosopher 2 is thinking
 Philosopher 1 is thinking
 Philosopher 5 is thinking
 Philosopher 3 is thinking
 Philosopher 4 is thinking

Philosopher 5 picked up the left chopstick
 Philosopher 2 picked up the left chopstick
 Philosopher 1 picked up the left chopstick
 Philosopher 4 picked up the left chopstick
 Philosopher 3 picked up the left chopstick



```
public class MainApp {  
  
    private static final int NBR_OF_PHILOSOPHERS = 5;  
  
    public static void main(String[] args) {  
  
        Philosopher[] philosophers = new Philosopher[NBR_OF_PHILOSOPHERS];  
        Object[] chopsticks = IntStream.range(0, NBR_OF_PHILOSOPHERS).mapToObj(i -> new Object()).toArray();  
  
        ExecutorService executor = Executors.newFixedThreadPool(NBR_OF_PHILOSOPHERS);  
        IntStream.range(0, NBR_OF_PHILOSOPHERS).forEach(i -> executor.submit(() -> {  
            Object leftChopstick = chopsticks[i];  
            Object rightChopstick = chopsticks[(i + 1) % NBR_OF_PHILOSOPHERS];  
  
            philosophers[i] = i == NBR_OF_PHILOSOPHERS - 1  
                ? new Philosopher(rightChopstick, leftChopstick, (i + 1))  
                : new Philosopher(leftChopstick, rightChopstick, (i + 1));  
  
            philosophers[i].start();  
        }));  
    }  
}
```

